MatShrink: Lossless Weight Compression for Back-to-Back Linear Layers in Transformers

Nils Graef[*], **Filip Makraduli**, **Siddh Shah**, **Siddharth Mohan**
OpenMachine

Abstract

We present MatShrink, a mathematically *exact* method for reducing the parameter count of any pair of consecutive linear layers. Unlike approximate compression techniques such as SVD-based rank reduction or magnitude pruning, MatShrink eliminates an $r \times r$ block of weights via an invertible change of basis on the shared inner dimension r, incurring zero accuracy loss by construction. We state and prove the core identity as a formal proposition and establish conditions under which the transformation is numerically stable. We specialize MatShrink to the V-O, Q-K, and latent projections of multi-head attention (MHA), multi-head latent attention (MLA), grouped-query attention (GQA), and multi-query attention (MQA), yielding lossless savings of 8–17% of per-layer attention parameters across representative models. We identify a bf16 numerical instability caused by ill-conditioned transformation matrices and prove that pivoted QR basis selection minimizes the worst-case condition number among all column-permutation strategies. Empirical validation on four models (GPT-2-124M, SmolLM2-135M, Gemma-3-1B, Gemma-3-4B) confirms exact fp32 equivalence and near-exact bf16 equivalence, with checkpoints loading directly in HuggingFace Transformers and vLLM without any runtime modification. MatShrink is applicable to both inference and training: Existing models can be retrofitted with MatShrink in a mathematically equivalent way, and future models can use MatShrink for both training and inference. We additionally propose a simplified MLA architecture that eliminates the separate RoPE projection cache, reducing both parameter count and long-sequence KV-cache overhead. See [1] for code and more transformer tricks.

For two back-to-back weight matrices W_A and W_B, Fig. 1 illustrates how MatShrink reduces the size of W_B in a mathematically equivalent way by using matrix inversion.

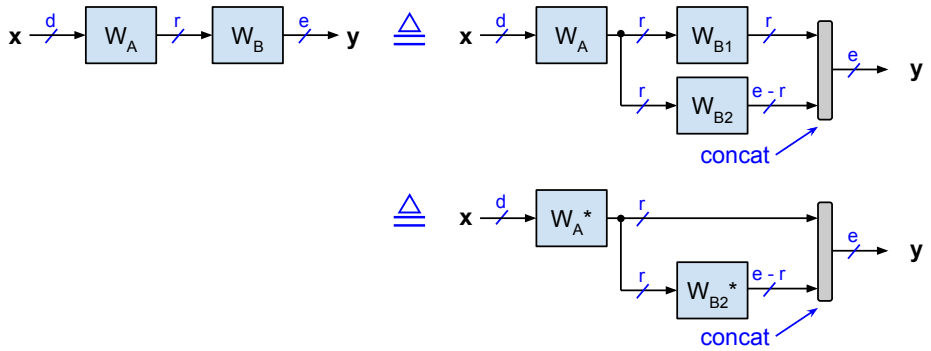

Figure 1: Mathematically equivalent implementations of two back-to-back weight matrices W_A and W_B with rank r, where $d > r$ and $e > r$, which reduces the total number of weights by r^2.

[*] info@openmachine.ai

Specifically, W_A is a $d \times r$ matrix, W_B is an $r \times e$ matrix with rank r, where $d > r$ and $e > r$. We can split W_B into two submatrices $W_{B1} \in \mathbb{R}^{r \times r}$ and W_{B2} such that $W_B = [W_{B1}, W_{B2}]$. We can then eliminate W_{B1} by merging it into W_A as $W_A^* = W_A W_{B1}$ and by changing W_{B2} to $W_{B2}^* = W_{B1}^{-1} W_{B2}$. This saves r^2 weights and r^2 multiply operations per token x. The following equation shows the mathematical identity of the modified back-to-back matrices, where I is the $r \times r$ identity matrix:

$$W = W_A \cdot W_B \ = W_A \cdot [W_{B1}, W_{B2}] \ = W_A W_{B1} \cdot \left[I, W_{B1}^{-1} W_{B2}\right] \ = W_A^* \cdot [I, W_{B2}^*]$$

Inverting the submatrix W_{B1} requires that this submatrix is invertible, which is often the case because it is extremely rare for large matrices to be non-invertible. In the rare case that W_{B1} is non-invertible, we can first permute the columns of the original matrix W_B such that the first r columns of the permuted matrix form an invertible submatrix W_{B1}.

For completeness, if $e = r$, then the entire matrix W_{B2} is eliminated. In general, if $e \leq r$, then the two matrices W_A and W_B are fused into a single matrix $W^* = W_A \cdot W_B$ with only de weights (instead of $dr + re$ weights for the original matrices W_A and W_B). This type of weight fusion is utilized in [2, 3].

Alternative way. Alternatively, we can split matrix W_A into two submatrices $W_{A1} \in \mathbb{R}^{r \times r}$ and W_{A2} such that $W_A = [W_{A1}; W_{A2}]$. We can then eliminate W_{A1} as $W = [W_{A1}; W_{A2}]\, W_B = [I; W_{A2}^*]\, W_B^*$ with identity matrix $I \in \mathbb{R}^{r \times r}$ and where $W_B^* = W_{A1} W_B$ and $W_{A2}^* = W_{A2} W_{A1}^{-1}$, see Fig. 2. This also saves r^2 weights and r^2 multiply operations per token x.

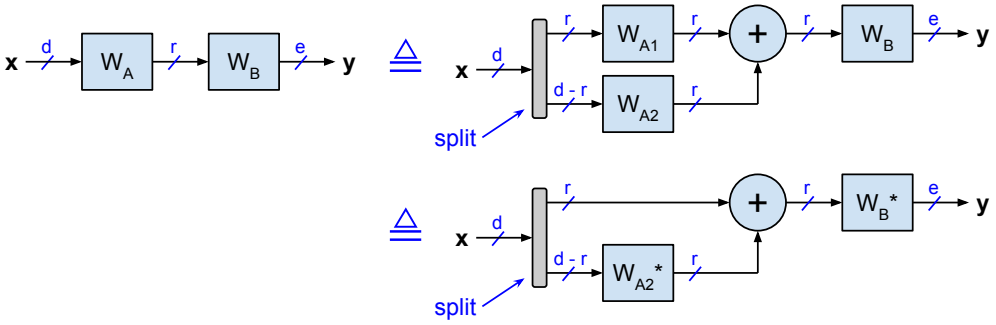

Figure 2: Alternative way of shrinking W_A instead of W_B

MatShrink for transformers. MatShrink reduces the number of weights for the following back-to-back weight matrices in transformer models:

- The V (value) and O (output) projections for each attention-head, see Fig. 3 and 4
- The Q (query) and K (key) projections for each attention-head (without the RoPE portion), see Fig. 5 and 6
- The latent projections of MLA (multi-head latent attention), see Fig. 7(b)

Related work. Many weight compression schemes for transformers have been proposed such as [4] and [5]. However, these schemes approximate the original weight matrices by using SVD (singular value decomposition) or other approximations. MatShrink on the other hand is not an approximation but an exact, mathematically equivalent optimization for back-to-back matrices. Furthermore, MatShrink is similar to slim attention [6] in its use of matrix inversion to compute projections from each other.

1 MatShrink for MHA transformers

MatShrink for V and O projections. Note that the value (V) and output (O) projections for each head i of multi-head attention (MHA) are two back-to-back weight matrices $W_{V,i}$ and $W_{O,i}$ as illustrated in Fig. 3 for a single attention head. Fig. 3 shows how MatShrink eliminates r^2 weights from the original O projection weight matrix W_O. Alternatively, Fig. 4 illustrates how the alternative

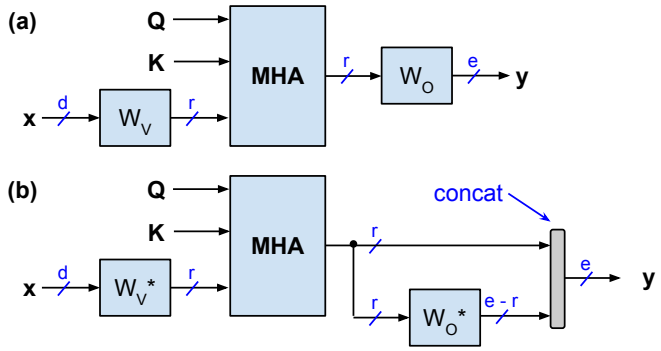

Figure 3: MatShrink for V and O projections of a single attention head: (a) original V and O projections; (b) equivalent implementation using MatShrink, which eliminates r^2 weights from W_O.

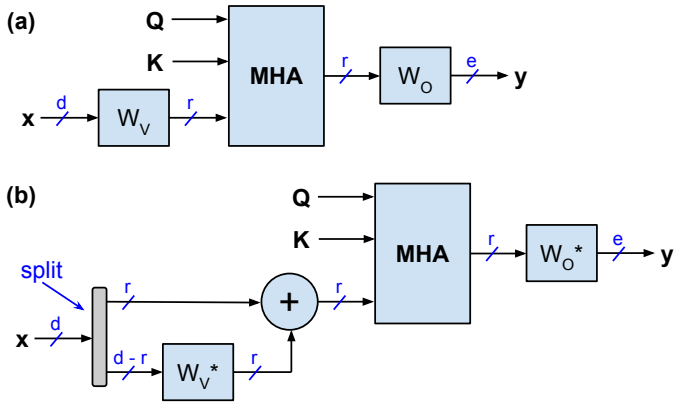

Figure 4: Alternative MatShrink implementation for V and O projections of a single attention head: (a) original V and O projections; (b) equivalent implementation using the alternative MatShrink scheme from Fig. 2, which eliminates r^2 weights from W_V.

MatShrink scheme from Fig. 2 can eliminate r^2 weights from the original V projection (instead of the O projection).

For MHA, we can apply the MatShrink scheme to each head. Specifically:

- For the vanilla MHA with h heads, each head has dimension $d_k = d/h$, and $d = d_{\text{model}}$.
- So for the dimensions r and e of Fig. 1, we have $r = d/h$ and $e = d$.
- This saves $r^2 = d^2/h^2$ weights for each head, so d^2/h weights in total.
- Note that for single-head attention (where $h = 1$), we can save $2d^2$ weights: Specifically, we can merge the V and O weight matrices into a single $d \times d$ matrix; and the Q and K weight matrices into a single $d \times d$ matrix (if there is no RoPE).

Table 1 lists the configurations of various MHA transformer models, the number of weights for their attention projections, and the weight savings provided by MatShrink.

Table 1: Weight savings for MHA transformer models using MatShrink

Model	d	d_k	h	weights $d \times (d_k h)$	savings $d_k^2 h$	savings %
Whisper-tiny [7]	384	64	6	147K	25K	17%
CodeGemma-7B [8]	3,072	256	16	12.6M	1.0M	8%
T5-3B [9]	1,024	128	32	4.2M	0.5M	12%
T5-11B [9]	1,024	128	128	16.8M	2.1M	13%

MatShrink for Q and K projections. For models that don't use RoPE (such as Whisper [7] and T5 models [9]), the query (Q) and key (K) projections for each head i of MHA are two back-to-back weight matrices $W_{Q,i}$ and $W_{K,i}$ as illustrated in Fig. 5 for a single attention head.

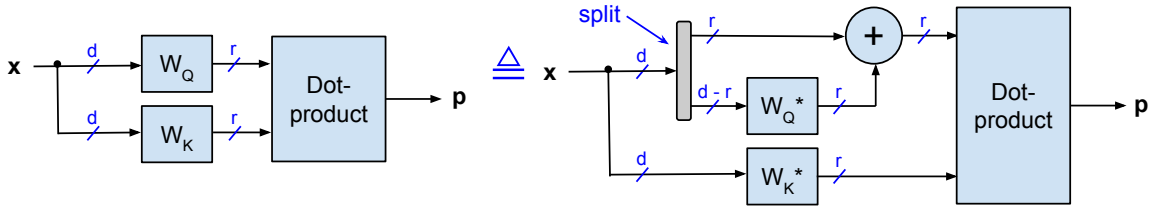

Figure 5: MatShrink for Q and K projections of a single attention head. Left: original Q and K projections and their dot-product p. Right: equivalent implementation using MatShrink, which eliminates r^2 weights from W_Q.

Fig. 5 shows how MatShrink eliminates r^2 weights from the original Q weight matrix W_Q. Note that the queries Q^* and keys K^* generated by the modified linear layers W_Q^* and W_K^* are not identical to the original queries Q and keys K, but their dot-products p are identical, i.e. $p = Q \cdot K = Q^* \cdot K^*$.

For many models that use RoPE, we can also apply this trick as follows: Many modern transformer models use partial RoPE, which applies RoPE to only a portion of the head-dimension $d_k = d/h$, usually only to one half of d_k. So in this case $r = d_k/2 = d/(2h)$, which saves only $r^2 = d^2/(4h^2)$ weights for each head, so $d^2/(4h)$ weights in total.

For completeness, Fig. 6 illustrates an alternative implementation of MatShrink that removes r^2 weights from W_K instead of W_Q.

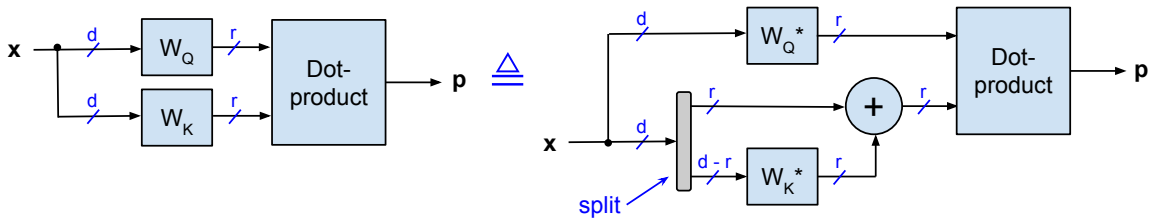

Figure 6: Alternative MatShrink implementation for Q and K projections of a single attention head. Left: original Q and K projections and their dot-product p. Right: equivalent implementation using MatShrink, which eliminates r^2 weights from W_K (instead of W_Q).

2 MatShrink for MLA transformers

Table 2 shows the configurations of various transformer models with MLA. We are using the following parameter names similar to [10]:

- For Q (query):
  - r_{Q}: rank of Q-latent projection
  - W_{DQ}: down-projection for Q
  - W_{UQ}: up-projection for Q-part without RoPE (aka NoPE)
  - W_{QR}: up-projection for Q-part with RoPE
- For KV (key-value):
  - r_{KV}: rank of KV-latent projection
  - W_{KR}: projection for K-part with RoPE (has its own cache, used for all queries as MQA)
  - W_{DKV}: down-projection for KV
  - W_{UK}: up-projection for K-part without RoPE (aka NoPE)
  - W_{UV}: up-projection for V

Table 2: Configurations of various MLA models

Model	Params	d	r_{Q}	r_{KV}	h	d_{NOPE}	d_{ROPE}
Perplexity R1-1776, DeepSeek-R1, V3	685B	7,168	1,536	512	128	128	64
Mistral Large 3 675B	675B	7,168	1,536	512	128	128	64
DeepSeek-V2.5	236B	5,120	1,536	512	128	128	64
DeepSeek-V2-lite, VL2-small [10]	16B	2,048	N/A	512	16	128	64
OpenBMB MiniCPM3-4B [11]	4B	2,560	768	256	40	64	32

DeepSeek's MLA (multi-head latent attention) scheme [10] has two latent projections, one for Q (queries) and one for KV (keys and values). We can apply MatShrink to each of them:

- The Q-latent projection and query (Q) projections are two back-to-back weight matrices W_{DQ} and W_{UQ}.
- The KV-latent projection and key/value (KV) projections are two back-to-back weight matrices W_{DKV} and the union of W_{UK} and W_{UV}.

We can also apply MatShrink to each V-O head and the non-RoPE portion of the Q-K heads. Specifically, we can apply the MatShrink to the MLA weight matrices in the following order:

1. Apply MatShrink to the V-O weight matrices.
2. Apply MatShrink to the NoPE portion (i.e. the non-RoPE portion) of the Q-K weight matrices.
3. Apply MatShrink to the Q-latent projections. This step must be done after applying MatShrink to the Q-K weights.
4. Apply MatShrink to the KV-latent projections. This step must be done after applying MatShrink to the V-O weights.

Applying MatShrink to the KV-latent projections not only reduces weight matrices and corresponding compute, it can also reduce the compute complexity as follows, where r_{KV} is the rank of the KV-latent projections.

- Option 1: Use the r_{KV} neurons that don't require a weight matrix as keys. The number of those keys is $r_{\text{KV}}/d_{\text{NOPE}}$. Then these keys can be directly used for the softmax arguments, which saves some computation complexity.
- Option 2: Use the r_{KV} neurons as values (instead of keys). Then these values can be directly multiplied with the softmax scores, which saves some compute complexity.

TODO: details the savings, perhaps with a new table

3 Simplified MLA

In this section we propose a simplification for DeepSeek's MLA (multi-head latent attention).

Fig. 7 shows the K and V projections of MLA and the proposed simplification:

- Fig. 7(a) shows the MLA projections for K (keys) and V (values). Note that a single d_{ROPE} head is shared among all query-heads, where $d_{\text{ROPE}} = 64$ or 32 usually.
- Fig. 7(b) shows the mathematically equivalent version with MatShrink applied to the weight matrices W_{DKV} and W_{UK}.
- Fig. 7(c) shows the proposed simplified MLA scheme where the d_{ROPE} units (or channels) are sourced directly from the latent cache, instead of having a separate cache and W_{KR} for the d_{ROPE} units:
  - Note that this simplified scheme is not mathematically identical to the standard MLA scheme shown in Fig. 7(a).
  - The rank s of the simplified scheme could be larger than r (e.g. $s = r + d_{\text{ROPE}}$) or slightly lower than this (e.g. $s = r$).
  - Advantages include: If $s > r$, then there is more usable rank for the keys and values. So the cached latent space is better utilized. And if $s < r + d_{\text{ROPE}}$ then the total cache size is reduced.

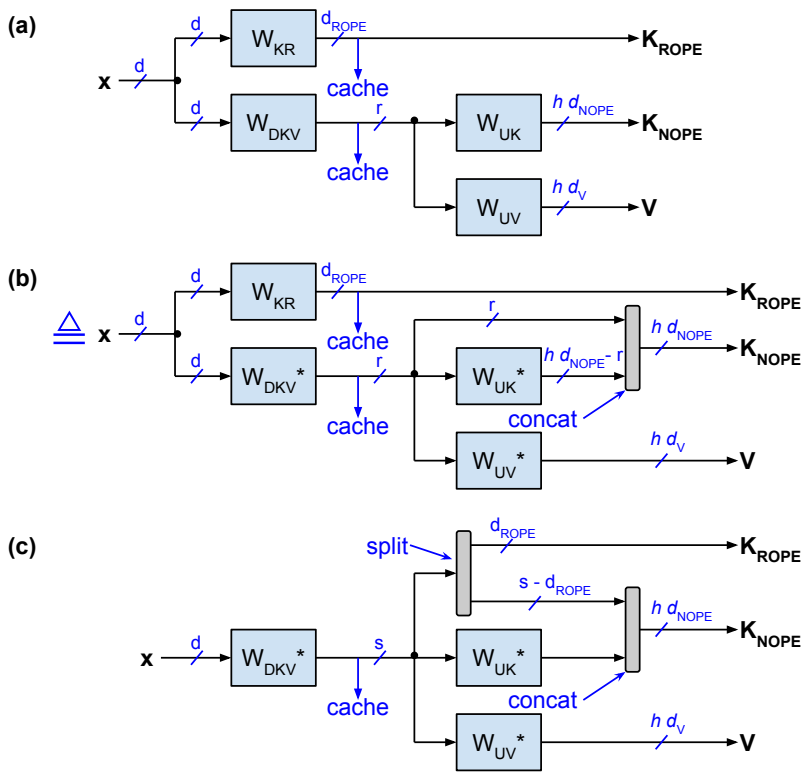

Figure 7: K and V projections for MLA. (a) original version; (b) equivalent version optimized by MatShrink; (c) proposed simplification

This simplification enhances MLA by directly leveraging the latent cache for RoPE components, potentially increasing effective rank and optimizing cache utilization. In DeepSeek-V2, standard MLA already reduces KV cache by compressing into latent vectors, but our proposal further streamlines this by eliminating separate RoPE projections, leading to additional memory savings especially for long-sequence inference.

4 MatShrink for GQA and MQA

MatShrink is not limited to MHA and MLA only. It's also applicable to GQA (grouped query attention) and MQA (multi-query attention). However, the savings are smaller than for MHA and MLA. Specifically, the savings are reduced by a factor g, where g is the number of queries that are shared among a single KV-pair, or in other words $g = n_{\text{heads}}/n_{\text{KV-heads}}$ (where n_{heads} is the number of query-heads, and $n_{\text{KV-heads}}$ is the number of KV-heads).

5 MatShrink for SVD

Approximate weight compression schemes such as [5] and [4] use SVD (singular value decomposition) to reduce the ranks of weight matrices, and thus reduce weights. This is applicable for example for the large weight matrices of the transformer's FFN (feedforward networks). The SVD decomposition factorizes the original matrix $W \in \mathbb{R}^{d \times e}$ into two matrices W_A and W_B where r is the compressed rank. After performing SVD and compressing the rank by a certain percentage, we can then eliminate r^2 weights using our MatShrink scheme. Note that reducing the rank by a certain percentage is not an exact implementation of the original matrix W but an approximation.

6 Experiments

We validate MatShrink's core claim of mathematical equivalence empirically on four models spanning both attention variants covered by the paper and both regimes of head dimension: SmolLM2-135M [12], a 30-layer decoder-only transformer with grouped-query attention ($n_{\text{heads}} = 9$, $n_{\text{KV}} = 3$, $d_{\text{head}} = 64$); GPT-2-124M [13], a 12-layer multi-head attention transformer ($n_{\text{heads}} = 12$, $d_{\text{head}} = 64$); and Gemma-3 [14] at the 1B (text-only) and 4B (multimodal, text-backbone-only) sizes, both with $d_{\text{head}} = 256$, the regime where MatShrink's per-head r^2 savings accumulate most (Table 1). All experiments run on a single NVIDIA A100 GPU and evaluate perplexity on the full WikiText-2 raw test split using non-overlapping context-length windows.

6.1 Correctness under fp32

We apply V-O MatShrink to every attention layer. For each KV group (GQA) or each attention head (MHA), we select the transformation matrix M and compute M^{-1} in double precision. Reconstruction error at the identity block is $\mathcal{O}(10^{-14})$ on every model (fp64 machine precision).

With shrunk weights stored in fp32, baseline and MatShrink perplexities match exactly on all four models (Table 3), confirming that the transformation is mathematically equivalent and that inversion-induced numerical error is negligible at single precision.

Table 3: WikiText-2 perplexity before and after V-O MatShrink, fp32 storage, full test split.

Model	Source	MatShrink	Δ
SmolLM2-135M (GQA)	15.4629	15.4629	0.0000
GPT-2-124M (MHA)	29.9394	29.9394	0.0000
Gemma-3-1B (GQA)	14.1839	14.1838	-0.0001
Gemma-3-4B (GQA)	10.7664	10.7664	0.0000

6.2 Numerical precision at bf16 and the role of basis choice on the shared inner dimension

Problem. The precision issue is specific to bf16 storage. Under fp16 the identity-M variant is already essentially lossless (the fp16 rows of Table 5 show absolute regressions of only $+0.01$ ppl on SmolLM2-135M and $+0.06$ ppl on GPT-2-124M, both within benchmark noise); fp16's 10-bit mantissa carries enough precision to absorb the rewritten-weight magnitudes. bf16 trades 3 mantissa bits (10→7) for 3 additional exponent bits (5→8); it is the reduced mantissa that bites here. With the naive choice of M (the leading $r \times r$ sub-block of the target W_O slice), bf16 perplexity degrades substantially, and the degradation grows with head dimension: on $d_{\text{head}} = 64$ models the absolute ppl regression is $+1.51$ on SmolLM2-135M and $+18.94$ on GPT-2-124M; on $d_{\text{head}} = 256$ Gemma 3 it is $+25.29$ on the 1B variant (a qualitatively broken model) and $+8.98$ on the 4B variant. The root cause is that M^{-1} amplifies bf16 rounding error when M is ill-conditioned: the condition number of the naive M has a heavy tail on every model (up to $\kappa = 1.67 \times 10^5$ on GPT-2, $\kappa = 1.58 \times 10^5$ on Gemma-3-4B), and the entries of $W_V^* = W_V M$ and $W_{O_2}^* = M^{-1} W_{O_2}$ then span a range that bf16's 8-bit mantissa cannot faithfully represent. Error compounds across layers.

Fix: change of basis on the shared inner dimension, realized by pivoted QR. The two back-to-back matrices W_V and W_O meet along a shared r-dimensional axis (the head dimension, which is the inner dimension of the composite product). Applying any invertible change of basis $T \in \mathbb{R}^{r \times r}$ to this shared axis as $W_V \mapsto W_V T$ and $W_O \mapsto T^{-1} W_O$ leaves the composite $W_V W_O$ unchanged, so the model's output at every token position is preserved exactly. What T does change is *which* $r \times r$ sub-block of W_O ends up in MatShrink's leading position and therefore gets inverted as M. A pure permutation of this shared axis is the orthogonal special case, and because orthogonal transformations preserve singular values it leaves $\text{cond}(M)$ unchanged; a general invertible T reduces $\text{cond}(M)$ by one to three orders of magnitude on the models we test.

We select the change of basis implicitly by picking M directly as the best-conditioned $r \times r$ sub-block of W_O, identified by rank-revealing (pivoted) QR decomposition of W_O:

$$(Q, R, P) \leftarrow \text{QR}(W_O,\ \text{pivoting} = \texttt{True}), \qquad M \leftarrow W_O\big[\,:,\ P[:r]\,\big],$$

which, by classical argument, selects the r columns of W_O forming the best-conditioned leading block available. The corresponding explicit change of basis would be $T = M \cdot M_0^{-1}$ with $M_0 =$

$W_O[:, : r]$ the naive leading sub-block; running vanilla MatShrink on $(W_V T, T^{-1} W_O)$ produces the same W_V^* and W_O^* as picking M directly, and the direct approach is the one-line shortcut used in the implementation. Alternative choices (column-L_2 sort, random sub-block sampling) are valid changes of basis as well and give partial improvements; pivoted QR is used because its conditioning guarantee is the strongest and because it is a one-line call in `scipy.linalg`. The resulting model is mathematically identical to the naive variant at fp32, stores the same number of weights, and loads in stock HuggingFace Transformers and vLLM without modification; the identity block simply lives at the column indices $P[: r]$ rather than at the contiguous prefix $[0, r)$.

Result. Pivoted QR reduces condition numbers by 1–3 orders of magnitude at the tail (Table 4) and collapses the bf16 perplexity regression to within ± 0.05 of the source model's bf16 perplexity on every tested configuration (Table 5). The fp32 perplexities are unchanged, as required by the mathematical invariance of the transformation.

Table 4: Distribution of cond(M) across all (layer, head/KV-group) pairs. Pivoted QR reduces the tail by roughly two orders of magnitude at $d_{\text{head}} = 64$ and by two to three orders at $d_{\text{head}} = 256$, where the naive tail is largest.

Model	Strategy	median κ	95th pct. κ	max κ
SmolLM2-135M (d_{head}=64)	identity	436	4.9×10^3	9.87×10^3
	pivoted	26	86	104
GPT-2-124M (d_{head}=64)	identity	255	3.4×10^3	1.67×10^5
	pivoted	37	118	404
Gemma-3-1B (d_{head}=256)	identity	4.4×10^3	1.7×10^4	9.91×10^4
	pivoted	218	563	702
Gemma-3-4B (d_{head}=256)	identity	2.6×10^3	2.4×10^4	1.58×10^5
	pivoted	156	570	5.4×10^3

Table 5: WikiText-2 perplexity on full test split, with pivoted-QR M-selection vs. naive identity-slice M-selection. fp32 is mathematically invariant (identical to four decimal places). Under bf16, pivoted-QR closes the gap to the source model from as large as $+25.29$ absolute (Gemma-3-1B identity-M) to within ± 0.05 on every model. Gemma 3 does not support fp16 inference on the source model (dynamic-range overflow), so only fp32 and bf16 are reported for those rows.

Model	Precision	Source	Identity-M	Pivoted-M
SmolLM2-135M	fp32	15.4629	15.4629	15.4629
	fp16	15.4634	15.4775	15.4634
	bf16	15.4750	16.9885	**15.4787**
GPT-2-124M	fp32	29.9394	29.9394	29.9394
	fp16	29.9482	30.0038	29.9467
	bf16	30.2090	49.1541	**30.1928**
Gemma-3-1B	fp32	14.1839	14.1838	14.1839
	bf16	14.1932	39.4825	**14.1520**
Gemma-3-4B	fp32	10.7664	10.7664	10.7664
	bf16	10.7877	19.7706	**10.8055**

6.3 vLLM compatibility

Because MatShrink in explicit-identity mode produces a standard checkpoint (identical tensor shapes and key names as the source), the shrunk model loads in vLLM [15] without modification to the runtime or model code. We verify end-to-end inference on SmolLM2-135M by running deterministic greedy generation before and after MatShrink; both produce coherent continuations from held-out prompts. The pivoted-QR variant is equivalent in this respect: the identity block lives at non-contiguous column indices, but every tensor retains its original shape and name, so no loader changes are required.

Because the identity blocks are stored as dense tensors, the runtime performs redundant matmuls and throughput is unchanged; realizing the compute savings implied by the identity structure requires runtime support, which we leave to future work.

Note on further quantization. A separate consequence of pivoted-QR selection is that W_V^* and W_O^* retain the magnitude range of the source W_V and W_O (since a well-conditioned M^{-1} does not inflate entries), whereas the naive identity-M variant produces rewritten weights with inflated magnitudes that track $\kappa(M)$. Downstream 8-bit or 4-bit weight quantization, which depends on a bounded weight distribution, should therefore compose more gracefully with pivoted-QR MatShrink than with the identity variant. Systematic evaluation of MatShrink plus quantization is left to future work.

7 Conclusion

TODO, also add experimental results

References

[1] OpenMachine. Transformer tricks. 2024. URL https://github.com/OpenMachine-ai/transformer-tricks.

[2] Nils Graef. KV-weights are all you need for skipless transformers. April 2024. *arXiv:2404.12362*.

[3] Marko Karbevski and Antonij Mijoski. Key and Value weights are probably all you need: On the necessity of the Query, Key, Value weight triplet in encoder-only and decoder-only transformers. January 2026. *arXiv:2510.23912*.

[4] Chi-Heng Lin, Shangqian Gao, James Seale Smith, Abhishek Patel, Shikhar Tuli, Yilin Shen, Hongxia Jin, and Yen-Chang Hsu. MoDeGPT: Modular Decomposition for Large Language Model Compression. 2024. *arXiv:2408.09632*.

[5] Pratyusha Sharma, Jordan T Ash, and Dipendra Misra. The truth is in there: Improving reasoning in language models with LAyer-SElective Rank reduction. December 2023. *arXiv:2312.13558*.

[6] Nils Graef and Andrew Wasielewski. Slim attention: cut your context memory in half without loss – K-cache is all you need for MHA. March 2025. *arXiv:2503.05840*.

[7] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. December 2022. *arXiv:2212.04356*.

[8] CodeGemma Team, Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A Choquette-Choo, Jingyue Shen, Joe Kelley, Kshitij Bansal, Luke Vilnis, Mateo Wirth, Paul Michel, Peter Choy, Pratik Joshi, Ravin Kumar, Sarmad Hashmi, Shubham Agrawal, Zhitao Gong, Jane Fine, Tris Warkentin, Ale Jakse Hartman, Bin Ni, Kathy Korevec, Kelly Schaefer, and Scott Huffman. CodeGemma: Open Code Models Based on Gemma. June 2024. *arXiv:2406.11409*.

[9] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. October 2019. *arXiv:1910.10683*.

[10] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, et al. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. *arXiv:2405.04434*.

[11] Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, Qianyu Chen, Huarong Zhou, Zhensheng Zou, Haoye Zhang, Shengding Hu, Zhi Zheng, Jie Zhou, Jie Cai, Xu Han, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. MiniCPM-V: A GPT-4V Level MLLM on Your Phone. 2024. *arXiv:2408.01800*.

[12] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Lewis Tunstall, Agustín Piqueres, Andres Marafioti, Cyril Zakka, Leandro von Werra, and Thomas Wolf. SmolLM2 - with great data, comes great performance, 2024. URL https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B.

[13] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. 2019.

[14] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, et al. Gemma 3 Technical Report. March 2025. *arXiv:2503.19786*.

[15] vLLM Project. vLLM. URL `https://github.com/vllm-project/vllm`.